Project Document - Ramesh Pragada

1. Project Overview

This project implements a Natural Language Processing (NLP) application for next-word prediction using Shakespeare's Hamlet from the NLTK Gutenberg corpus. The text is converted into numerical sequences, transformed into fixed-length training examples, and used to train a recurrent neural network. The attached Colab notebook implements an LSTM-based model with an Embedding layer and a softmax output layer. A custom prediction function accepts user text and predicts the next word.

2. Project Objectives

Load Shakespeare's Hamlet using the NLTK Gutenberg corpus.

Preprocess and clean the raw text.

Tokenize the text and create a word index.

Generate N-Gram sequences for supervised learning.

Prepare fixed-length input sequences and target labels.

Apply sequence encoding and categorical target representation.

Build and train an LSTM model for next-word prediction.

Save the best model checkpoint during training.

Create a custom function for predicting the next word from user input.

Evaluate the implemented training behavior and document limitations and future enhancements.

3. Dataset

The project uses the Shakespeare's Hamlet text available through NLTK's Gutenberg corpus. The notebook downloads/accesses the Gutenberg corpus and loads the file 'shakespeare-hamlet.txt' using gutenberg.raw().

4. Data Preprocessing

The notebook applies a simple text-cleaning step before tokenization. New-line characters, carriage returns, byte-order marks, and apostrophes are removed. This creates a continuous text representation for tokenization.

5. Tokenization and Vocabulary

Keras Tokenizer is used to convert the cleaned text into integer word IDs. Each unique token receives an integer index through tokenizer.word_index.

N-Gram Sequence Creation

The notebook uses a three-word input window to predict the fourth word. A sliding window is applied over the encoded text. For each position, four consecutive token IDs are collected: the first three become the input and the fourth becomes the target.

```
#creating a list and adding the words into the list
sequence = []
for i in range (3, len(sequence_data)):
  words = sequence_data[i-3: i+1]
  sequence.append(words)
```

The notebook produces 28,761 four-token sequences. Consequently, X contains 28,761 samples with three input tokens per sample, while Y contains the corresponding next-word target.

7. Target Encoding

The target word IDs are converted to one-hot encoded vectors using to_categorical(). This matches the softmax output layer, where the model produces a probability for each vocabulary word.

```
#Creating predictors by assigning array values to 'X', Last word to 'y'
X= []
Y= []

for i in sequence:
  X.append(i[0:3])
  Y.append(i[3])
```

```
X= np.array(X)
Y= np.array(Y)
```

```
y = to_categorical(Y, num_classes=vocab_size)
```

```
y[:5]
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

8. LSTM Model Architecture

The implemented model is a Sequential neural network consisting of an Embedding layer, two stacked LSTM layers, a dense hidden layer with ReLU activation, and a final vocabulary-sized softmax layer.

```
#Building the LSTM Model

model = Sequential()

model.add(Embedding(vocab_size, 10, input_length =3))
model.add(LSTM(1000, return_sequences = True))
model.add(LSTM(1000))
model.add(Dense(1000, activation = 'relu'))
model.build(input_shape=(None, 3))
model.add(Dense(vocab_size, activation= "softmax"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 3, 10)	54,490
lstm (LSTM)	(None, 3, 1000)	4,044,000
lstm_1 (LSTM)	(None, 1000)	8,004,000
dense (Dense)	(None, 1000)	1,001,000
dense_1 (Dense)	(None, 5449)	5,454,449

```
Total params: 18,557,939 (70.79 MB)
Trainable params: 18,557,939 (70.79 MB)
Non-trainable params: 0 (0.00 B)
```

9. Model Compilation and Training

The model is compiled with categorical crossentropy loss and the Adam optimizer using a learning rate of 0.001. Training is performed for two epochs with a batch size of 64. A ModelCheckpoint callback saves the model whenever the monitored training loss improves.

```
#Compiling the next-word prediction neural network using categorical crossentropy and Adam optimizer
from tensorflow.keras.callbacks import ModelCheckpoint
checkpoint=ModelCheckpoint("next_words.h5", monitor= "loss", verbose =1, save_best_only=True)
model.compile(loss="categorical_crossentropy",optimizer= Adam(learning_rate = 0.001))
model.fit(X,y, epochs=2, batch_size = 64, callbacks=[checkpoint])
```

```
Epoch 1/2
450/450 ━━━━━━━━━━━━━━━━━━━━ 0s 677ms/step - loss: 7.1648
Epoch 1: loss improved from None to 6.93918, saving model to next_words.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`.

Epoch 1: finished saving model to next_words.h5
450/450 ━━━━━━━━━━━━━━━━━━━━ 310s 678ms/step - loss: 6.9392
Epoch 2/2
450/450 ━━━━━━━━━━━━━━━━━━━━ 0s 656ms/step - loss: 6.5473
Epoch 2: loss improved from 6.93918 to 6.53442, saving model to next_words.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`.

Epoch 2: finished saving model to next_words.h5
450/450 ━━━━━━━━━━━━━━━━━━━━ 312s 657ms/step - loss: 6.5344
<keras.src.callbacks.history.History at 0x7d0ca8a0be00>
```

11. Next-Word Prediction Function

After training, the notebook reloads the saved model and tokenizer. The custom predict_word() function converts the supplied text into token IDs, obtains the model's predicted class using argmax, and maps the predicted index back to a word using tokenizer.word_index.

```python
#Loading the model and Creating a function to check the word prediction
from tensorflow.keras.models import load_model

model = load_model("next_words.h5")
tokernizer = pickle.load(open('token.pkl','rb'))

def predict_word(model, tokenizer, text):
  sequence = tokenizer.texts_to_sequences([text])
  sequence = np.array(sequence)
  preds=np.argmax(model.predict(sequence))

  predicted_word=""
  for key, value in tokenizer.word_index.items():
    if value == preds:
        predicted_word = key
        break
  print(predicted_word)
  return predicted_word

while True:
  text = input("Enter your line: ")
  if text == "1":
    break
  else:
    text = text.split(" ")
    text = text[-3:]
    predict_word(model, tokenizer,text)
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
Enter your line: I thinke I heare
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 405ms/step
the
Enter your line: Tis now strook twelue, get
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 568ms/step
and
Enter your line:
```

12. Conclusion

The project demonstrates an end-to-end next-word prediction pipeline using Shakespeare's Hamlet and a recurrent neural network. The text is transformed from raw Gutenberg content into token IDs and sliding N-Gram training examples. A stacked LSTM model learns from three-word contexts and predicts the probability distribution of the following word. The attached execution achieved a reduction in training loss from 6.93918 to 6.53442 over two epochs, and the custom prediction interface successfully returned a next-word prediction for a sample input.

For full alignment with the stated project requirements, the next version should add a GRU model, Dropout, EarlyStopping, a proper validation/test split, quantitative evaluation, and a direct LSTM-versus-GRU comparison.

13. Key Libraries and Technologies

Technology	Purpose
Python	Programming and data-processing language
NLTK Gutenberg	Source of Shakespeare's Hamlet text
TensorFlow / Keras	Deep learning framework
Keras Tokenizer	Text tokenization and word indexing
Embedding	Dense representation of token IDs
LSTM	Recurrent neural network for sequence learning
NumPy	Numerical array manipulation
Pickle	Tokenizer serialization
Adam	Optimization algorithm